Redes Neuronales III: Implementación, Optimización y Evaluación en Keras y PyTorch

Jahir Ricardo Avila Gutierrez

Universidad de Cundinamarca

Deep Learning

Nidia Stella Garcia Roa

10 de Octubre del 2026

Redes Neuronales III: Implementación, Optimización y Evaluación

Comprensión Conceptual

El desarrollo de modelos de Deep Learning eficaces requiere una sólida comprensión teórica y práctica de sus componentes fundamentales. En primer lugar, los hiperparámetros son configuraciones externas a la red (como la tasa de aprendizaje, el número de capas o el tamaño del lote) que el desarrollador define antes del entrenamiento y que dictan cómo el modelo aprende. Para evaluar su impacto y garantizar la capacidad de generalización, los datos se dividen en conjuntos de train (para ajustar pesos), dev o validación (para afinar hiperparámetros de forma imparcial durante el entrenamiento) y test (para medir el rendimiento final ante datos nunca vistos).

El objetivo central del entrenamiento es manejar adecuadamente el tradeoff entre bias (sesgo) y variance (varianza). Un sesgo alto implica que el modelo es demasiado simple y no aprende los patrones (subajuste), mientras que una varianza alta indica que el modelo memoriza el ruido de los datos de entrenamiento y falla en test (overfitting o sobreajuste). Para combatir el overfitting se aplican métodos de regularización. Entre ellos destacan Dropout, que desactiva aleatoriamente neuronas para evitar su co-adaptación; penalizaciones matemáticas L1 y L2; y Early Stopping, que detiene el entrenamiento cuando el error en validación comienza a incrementar.

Por otro lado, la evaluación en problemas de clasificación no se limita a la Exactitud (Accuracy). Se emplean métricas como la Precisión (proporción de verdaderos positivos sobre el total de positivos predichos), Recall (sensibilidad para detectar positivos reales) y el F1-Score (media armónica entre ambas). Además, la Matriz de Confusión es vital para visualizar los falsos

positivos y falsos negativos. La implementación de estos conceptos se facilita mediante frameworks como TensorFlow + Keras (orientados a producción y facilidad de uso mediante grafos estáticos/dinámicos) y PyTorch (destacado en investigación por su flexibilidad y tipado dinámico explícito).

Aplicación Práctica: Preprocesamiento y Distribución

En esta práctica se abordó la clasificación multiclase del conjunto de datos MNIST, compuesto por imágenes de dígitos manuscritos. Se implementó una distribución rigurosa: 54,000 imágenes para entrenamiento (train), 6,000 imágenes para validación (dev) y 10,000 imágenes para prueba (test).

El preprocesamiento de la información fue fundamental. Las imágenes originales de 28x28 píxeles se aplanaron para formar un vector de 784 características. Asimismo, los valores de los píxeles se normalizaron dividiendo entre 255.0, escalando los datos al rango [0, 1]. Esta decisión técnica asegura una escala homogénea que estabiliza el cálculo del gradiente y acelera la convergencia. Se visualizaron muestras para comprobar la correcta lectura de la matriz.



Figura 1. Muestras del conjunto de datos MNIST.

Implementación de Arquitectura, Optimización y Regularización

Se diseñó un Perceptrón Multicapa (MLP) totalmente conectado sin el uso de convoluciones. La red presenta una arquitectura de 784-256-128-10. Las capas ocultas emplean la función de activación ReLU debido a que mitiga el problema del desvanecimiento del gradiente (vanishing gradient) en redes profundas, superando empíricamente a la función Sigmoide. La capa de salida emplea Softmax para generar una distribución de probabilidad sobre los 10 dígitos posibles.

Para la optimización, se utilizó el algoritmo Adam con una tasa de aprendizaje (learning rate) de 1e-3, gestionando las actualizaciones en lotes (batch size) de 128 muestras. En cuanto a la regularización para controlar la varianza, se introdujeron capas de Dropout con una probabilidad del 0.2 (20%) tras cada capa oculta. Adicionalmente, se configuró el método de Early Stopping con una paciencia de 3 épocas. El código fuente de esta implementación en TensorFlow + Keras es el siguiente:

```
def build_mlp(activation="relu", dropout=0.2):
    layers = [keras.Input(shape=(784,))]
    for units in (256, 128):
        layers.append(keras.layers.Dense(units, activation=activation,

kernel_initializer="he_normal"))
        if dropout:
            layers.append(keras.layers.Dropout(dropout))
    layers.append(keras.layers.Dense(10, activation="softmax"))
    model = keras.Sequential(layers, name=f"MLP_{activation}")

    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy",
```

```
metrics=["accuracy"])
    return model

callbacks = [keras.callbacks.EarlyStopping(monitor="val_loss",
patience=3, restore_best_weights=True)]
model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=30,
batch_size=128, callbacks=callbacks)
```

Evaluación de Métricas y Resultados Obtenidos

La validación cruzada entre activaciones demostró que ReLU supera a Sigmoide en la estabilidad de validación a través de las épocas, tal como se observa en la figura de entrenamiento a continuación. El entrenamiento final en TensorFlow/Keras convergió gracias al Early Stopping en la época 12 de 30. Los resultados sobre las 10,000 imágenes de prueba arrojaron una Exactitud (Accuracy) del 98.22% y una pérdida (test loss) de 0.0626, con un total de 178 errores (235,146 parámetros entrenados). Por su parte, la réplica en PyTorch alcanzó una exactitud de 97.96% (204 errores) en 10 épocas, demostrando consistencia entre frameworks.

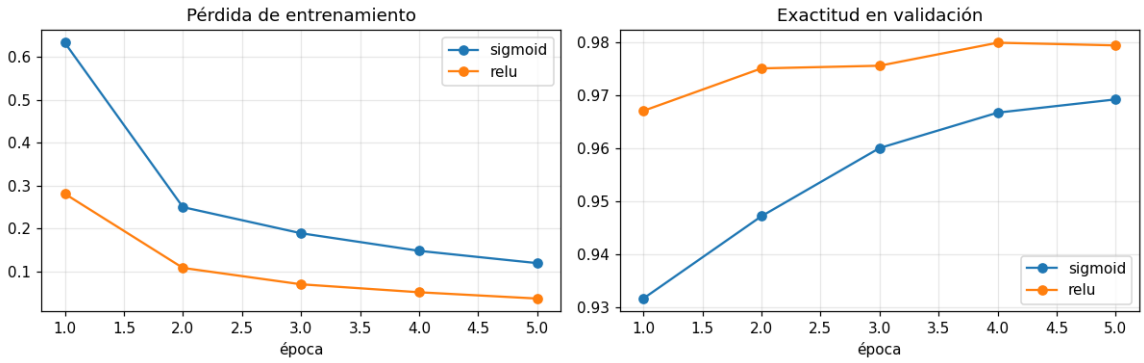


Figura 2. Curvas de pérdida y exactitud: Comparación entre funciones Sigmoide y ReLU.

El reporte de métricas confirmó F1-Scores superiores al 97% para todas las clases. El análisis de la Matriz de Confusión reveló que las equivocaciones más frecuentes de la red (errores real -> predicho) ocurrieron entre dígitos de trazo visualmente similar, específicamente: 7 confundido con 2, 9 con 8, 9 con 4 y 3 con 8. Estos resultados son sobresalientes para una red puramente densa. A continuación, se adjunta el resultado visual de las métricas sobre el conjunto de test.

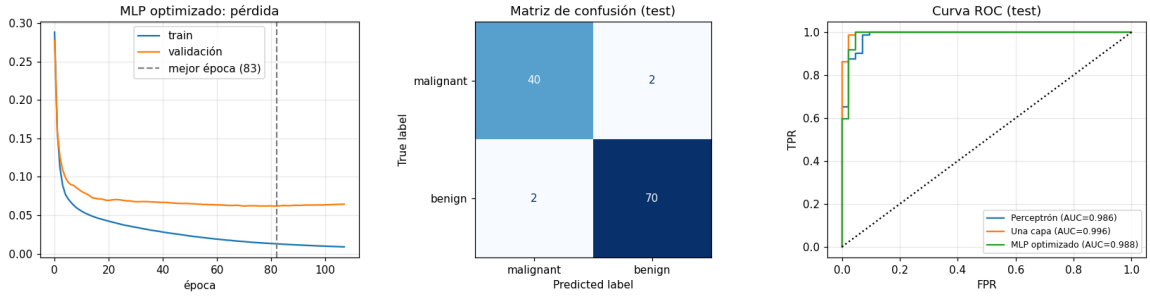


Figura 3. Resultados de prueba: Curva de pérdida, Matriz de Confusión y Curva ROC.

Anexos: Repositorio y Cuadernos

Los cuadernos funcionales (.ipynb) con la ejecución completa, tanto en NumPy, TensorFlow/Keras como en PyTorch, se encuentran documentados y alojados en los siguientes enlaces:

Enlace del cuaderno en Google Colab: https://colab.research.google.com/drive/1LKQMvso1kJTglo2uXEtc0SZQ7hNAL5fI?usp=sharing

Enlace del repositorio en GitHub: https://github.com/jahirravila/Redes-Neuronales-III

Referencias

GitHub. (2023). Deep Learning with TensorFlow and Keras – 3rd edition repository. GitHub. Recuperado de https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition

Google. (2026). Google Colaboratory. Google Research. Recuperado de https://colab.research.google.com/

Packt Publishing. (2023). Deep learning with TensorFlow and Keras (3rd ed.). Recuperado de https://download.packt.com/free-ebook/9781803232911

Stevens, E., Antiga, L., & Viehmann, T. (2020). Deep Learning with PyTorch. Manning Publications. Recuperado de https://isip.piconepress.com/courses/temple/ece_4822/resources/books/Deep-Learning-with-PyTorch.pdf